FirstKnock Sales OS — Technical Audit Report

FirstKnock Sales OS — Technical Audit Report

Production-Grade Codebase Analysis & Remediation Roadmap

April 2026

Prepared By	Senior Software Architect
Date	April 2026
Repository	invisibleontheblockchain/firstknock-sales-os
Codebase Size	~400+ source files, 37 cloud functions, React/Vite/Capacitor frontend
Stack	React 18.2.0 · Vite 6.1.0 · TypeScript 5.8.2 · Capacitor 8.0.0 · Deno Cloud Functions · Base44 BaaS
Data Providers	RentCast API · BatchData API · H3 Spatial Index (Uber) · Neon PostgreSQL

1. Executive Summary

Overall Code Quality Assessment

The FirstKnock Sales OS is an architecturally ambitious geospatial lead generation platform that demonstrates genuine engineering sophistication in its core data pipeline. The three-step property validation pipeline — RentCast dual-stream aggregation, heuristic DOM scoring, and BatchData synchronous verification — reflects a mature understanding of real estate data quality problems and API cost economics. The self-chaining chunk processor in processFetchChunk/entry.ts, with its mutex-based idempotency guard and exponential backoff with jitter, is production-quality infrastructure. The Beta-Binomial Bayesian ML model (bayesian_v3) with ADWIN drift detection, the 2-opt TSP route optimizer with street-sweep pattern, and the H3 resolution-9 spatial indexing

all indicate a team capable of building non-trivial systems.

However, the codebase exhibits a pattern common to fast-moving startups: production-critical infrastructure built alongside one-off scripts and prototype code that was never cleaned up. The most dangerous artifact is fixChristianRoute/entry.ts — a repair script containing hardcoded PII (christian@nativapest.com), no authentication check, and the ability to mutate up to 25,000 records while triggering unbounded BatchData API costs. This file is not a theoretical risk; it is an unauthenticated HTTP endpoint in production. Similarly, processReferral/entry.ts contains a catastrophic O(n) database scan (User.list('-created_date', 5000)) that will cause out-of-memory crashes as the user base grows, and a TOCTOU race condition in commission crediting that will produce financial discrepancies under concurrent load.

The system's offline-first mobile architecture using localforage/IndexedDB is well-conceived but stores unencrypted PII at rest, violating CCPA/GDPR mobile security hygiene. The address hashing strategy suffers from a split-brain problem: three incompatible hash formats exist across processFetchChunk, generateOptimizedRoutes, and migrateHashLegacy, guaranteeing cache misses and duplicate records when the same property traverses different pipeline paths. The GPS proof-of-visit system, a core value proposition of the platform, has no server-side velocity validation — any rep can spoof their location with a mock GPS app. The overall assessment is: strong core architecture, critical production safety gaps, remediable within two focused engineering sprints.

Top 3 Critical Risks

P0 — IMMEDIATE ACTION REQUIRED: The following three issues represent active production risks that can cause financial damage, data corruption, or security breaches without any attacker sophistication.

Table 1 — Top 3 Critical Production Risks

Risk	Location	Severity	Description
fixChristianRoute in Production	fixChristianRoute/entry.ts	P0 — CRITICAL	Hardcoded PII (christian@nativapest.com), zero authentication, uses asServiceRole bypassing all RLS. Any unauthenticated HTTP request mutates up to 25,000 records and queues unbounded BatchData API calls. Errors silently swallowed via .catch(() => {}).

O(n) User Table Scan	processReferral/entry.ts	P0 — CRITICAL	User.list('-created_date', 5000) fetches the entire user table into memory to find a single referral code. Causes OOM crashes and timeouts at scale. Users beyond position 5,000 have codes that silently fail to apply.
GPS Spoofing Vulnerability	GpsTracker.jsx + knock verification	P1 — HIGH	No server-side velocity checks on consecutive knock coordinates. Reps can use Android Mock Location or iOS Xcode simulation to fake routes from home. No device attestation (Play Integrity / DeviceCheck) implemented.

Top 3 Quick Wins

1. Replace O(n) scan with indexed query: await base44.asServiceRole.entities.User.filter({ referral_code: code }) — estimated 2 hours, eliminates OOM risk entirely.
2. Delete or gate fixChristianRoute behind admin auth + remove hardcoded PII (christian@nativapest.com and ZIP codes ['29621','29624','29625','29626','29627']) — estimated 1 hour, eliminates the most dangerous attack surface.
3. Add server-side GPS velocity check: reject InteractionLog entries where Haversine distance between consecutive knocks divided by elapsed seconds exceeds 26.8 m/s (60 mph) — estimated 8 hours, closes the core knock-fraud vector.

2. Data Architecture Review

MasterProperty Schema Completeness

The MasterProperty entity is the central deduplicated property record, keyed on address_hash. The confirmed schema fields from decoded source analysis are comprehensive for the current use case, but several high-value fields are computed at runtime rather than persisted — a pattern that adds latency to every route generation and prevents efficient database-level filtering.

Table 2 — Confirmed MasterProperty Schema Fields

Field	Type	Source	Notes
address_hash	STRING (PK)	processFetchChunk	Pipe-separated: "123 MAIN ST|77001". Primary dedup key.
house_number, street_name, zip_code	STRING/INT	RentCast	Core address components
lat, lng	FLOAT	RentCast / geocodeProperties	Coordinates for Leaflet + H3 indexing
h3_index	STRING	geocodeProperties	Uber H3 hex at resolution 9 (~174m² cells)
price, sold_date, year_built, lot_size	INT/DATE	RentCast / BatchData	Core property attributes
original_status	STRING	Pipeline	SOLD | ELIGIBLE | HEURISTIC_SOLD | BATCHDATA_CONFIRMED | REJECTED
sale_confidence	ENUM	Pipeline	high | medium | low | verified | REJECTED
mlsConfidence	STRING	Heuristic engine	low | medium — BatchData eligibility gate
property_type	STRING	RentCast	Single Family | Townhouse| Condo | etc.
data_source	STRING	Pipeline	rentcast | batchdata_verified | csv_import
history	OBJECT	RentCast MLS	Listing history count used in heuristic scoring

Missing Fields That Would Unlock New Capabilities

- homeowner_equity — Currently estimated via 3% appreciation heuristic in scoreProperty(). Should be a real field sourced from BatchData deed records to enable accurate equity-based lead scoring.
- ownership_tenure_years — Derived from sold_date at runtime. Storing directly enables database-level filtering (e.g., 'owned > 5 years') without full table scans.
- neighborhood_velocity — H3 hex heat is computed at runtime via gridDisk(h3Index, 1). Persisting this field would reduce route generation latency significantly.

- contact_enrichment — No phone/email fields visible in the schema. Integration with a skip-tracing provider would unlock direct outreach workflows.
- days_since_last_knock — Computed from InteractionLog at runtime via O(ntimesm) join. Denormalizing this field would eliminate the filterByStreetCooldown() performance bottleneck.
- last_interaction_date — Denormalized from InteractionLog for faster filtering and sorting without cross-entity joins.
- competitor_activity_signals — No competitor tracking fields exist. Tracking which properties have been knocked by competing teams would improve territory strategy.

address_hash Normalization: The Split-Brain Problem

Three incompatible address hash formats exist across the codebase. When the same property is processed by different pipeline paths, it will generate different hashes — causing duplicate records, cache misses, and data integrity failures.

Table 3 — Address Hash Format Comparison

Hash Format	Location	Example Output	Risk
Pipe-separated (primary)	processFetchChunk/entry.ts — generateNormalizedHash()	"123 MAIN ST|77001"	Primary format. Uppercase, punctuation stripped, street types abbreviated (ST/AVE/BLVD).
Three-part pipe (dedup)	routeOptimizer.jsx — generateOptimizedRoutes()	"`${num}\|${street}|${zip}`"	No street type normalization. "123|Main Street|77001" neq "123 MAIN ST|77001". Causes route-level
Kebab-case (legacy)	migrateHashLegacy/entry.ts — buildAddressHash()	"123-Main-St-Houston-TX-77001"	100% cache miss rate against primary tables. Will create duplicate records if used for lookups.

Additional collision risks exist within the primary hash format itself. Unit numbers are not normalized: '123 MAIN ST APT 4|77001' and '123 MAIN ST UNIT 4|77001' produce different hashes for the same physical unit. Directional prefixes are not stripped: '123 N MAIN ST|77001' and '123 MAIN ST|77001' are treated as different properties. The fix requires standardizing on a single canonical hash function across all pipeline paths and migration scripts, with a one-time re-hash migration for existing records.

Table 4 — H3 Resolution 9 Analysis

Dimension	Detail
Cell Area	~174m² (0.105 km²) per hexagon at resolution 9
Physical Coverage	Approximately 1–3 city blocks per cell — appropriate for D2D neighborhood heat mapping
Usage in Codebase	latLngToCell(p.lat, p.lng, 9) in routeOptimizer.jsx; gridDisk(h3Index, 1) spreads heat to 7 adjacent cells
Street Boundary Issue	A single street may span 2–3 hexagons, causing heat scores to be split across cells rather than aggregated
Territory Enforcement	NOT used for territory boundaries. Territories use ray-casting isPointInPolygon() instead — H3 is only used for heat scoring
Recommendation	Consider resolution 8 (~0.74 km²) for territory-level heat aggregation alongside resolution 9 for property-level indexing

Indexing Strategy

From code analysis, the following fields are heavily filtered in production queries. Fields marked as missing indexes represent active performance risks — full table scans on these fields will cause query timeouts as data volume grows.

Table 5 — Critical Index Requirements

Field	Entity	Query Pattern	Index Status
address_hash	MasterProperty	Primary dedup key on every insert/update	Required — likely exists
zip_code	MasterProperty	Batch queries by ZIP in fetchZipProperties	Required — likely exists
status	FetchJob	filter({ status: 'running' }) on every pipeline poll	Required — must verify
user_email	FetchJob	Filtered per user on every dashboard load	Required — must verify

referral_code	User	Currently O(n) full table scan — NO index	MISSING — P0 fix required
address_hash	InteractionLog	Joined on every route generation for knock history	Required — must verify
assigned_to	SavedRoute	Filtered in autoAssignRoute and RepHome route discovery	Required — must verify
manager_id	SavedRoute	Filtered in autoAssignRoute top-100 pending query	Required — must verify

3. Three-Step Pipeline Analysis

Pipeline Overview

The three-step validation pipeline is the architectural centerpiece of FirstKnock. It executes synchronously before any record is written to MasterProperty, guaranteeing zero false-positive leads reach the map. Step 1 aggregates two parallel RentCast data streams (County Deeds via status=Active and MLS Inactive via status=Inactive), deduplicates them by address (deed wins over MLS), and normalizes addresses. Step 2 runs every MLS Inactive record through the heuristic DOM scoring engine. Step 3 sends ambiguous records to BatchData for paid verification, capped at 100 calls per chunk.

70–80%BatchData Cost Reduction via CDC Delta-Pull + Heuristic Gating	**85%**API Call Reduction for Re-Pulls (CDC High-Watermark Delta)	**100**Max BatchData Calls Per Chunk (Hard Cap)	**500ms**Self-Chain Delay Between Chunk Invocations

Heuristic Scoring Engine — Detailed Analysis

The DOM threshold scoring engine in processFetchChunk/entry.ts evaluates MLS Inactive listings to determine genuine sales versus contract expirations. The contract boundary auto-reject logic — Math.abs(dom - 90) <= 3 || Math.abs(dom - 180) <= 3 || Math.abs(dom - 365) <= 3 — catches the three standard MLS contract renewal boundaries with a plusminus3-day tolerance window. This is a sound heuristic: MLS contracts auto-expire at exactly these boundaries, and a removal on day 90 is statistically far more likely to be an expiration than a sale.

Table 6 — Heuristic Scoring Engine Rules (processFetchChunk vs fetchZipProperties)

Condition	processFetchChunk Score	fetchZipProperties Score	Rationale
Math.abs(dom - 90/180/365) <= 3	Auto-Reject	hScore -= 3	MLS contract renewal boundaries — almost certainly expiration, not sale
dom > 150	hScore -= 3	hScore -= 3	Stale listings rarely convert to sales
dom > 60	hScore -= 2	hScore -= 2	Moderately stale listing
listingDuration < 7	hScore -= 1	hScore -= 2	Flash listings are test posts or data entry errors — INCONSISTENCY
dom < 30	hScore += 3	N/A	Standard escrow close window
dom >= 30 && dom <= 45	N/A	hScore += 3	Standard escrow close window (ZIP path)
dom >= 14 && dom < 30	N/A	hScore += 2	Fast-moving market close (ZIP path)
hScore <= -4 or <= -2	heuristicRejected	—	Dropped — never enters database
hScore >= 3 AND dom < 30	mlsConfidence = 'medium'	—	BatchData eligible — fresh + high-confidence
hScore >= 1	mlsConfidence = 'low'	—	Written to DB, not BatchData-verified

INCONSISTENCY DETECTED: The listingDuration < 7 penalty is -1 in processFetchChunk/entry.ts but -2 in fetchZipProperties/entry.ts. The same property will receive a different heuristic score depending on which pipeline path processes it — a data integrity issue that can cause properties to be BatchData-verified via one path but rejected via another.

BatchData 100-Call Cap & Cost Architecture

The hard 100-call cap is implemented as: const allowedMisses = ambiguous.slice(0, 100). The 101st ambiguous record remains classified at mlsConfidence = 'low' without BatchData verification. This is the correct cost-control decision. Batches of 10 are sent with a 250ms sleep between groups: await new Promise(r => setTimeout(r, 250)). At 10 calls per batch with 250ms between batches, the theoretical throughput is 40 BatchData calls per second — well within BatchData's typical 100 req/s rate limit. The conservative sleep is appropriate for a serverless environment

where cold starts add latency variance.

DOCUMENTATIONDISCREPANCY: The FetchJob.jsonc schema comment states total_batchdata_calls is 'capped at 500'. The actual implementation in fetchZipProperties/entry.ts enforces a hard cap of 100 via allowedMisses.slice(0, 100). Either the schema must be updated to reflect 100, or the code must be updated to match the 500-call design intent. This discrepancy will mislead future engineers about the system's cost model.

Chunk Overlap Strategy & Gap Risk

The fetchAreaProperties/entry.ts function divides the drawn polygon into overlapping sub-circles arranged in a hex-grid pattern. The sub-circle radius is 5 miles with an 80% overlap factor, producing an 8-mile step between circle centers (SUB_CIRCLE_RADIUS * 2 * OVERLAP_FACTOR = 8). The 20% overlap zone ensures boundary properties are captured by at least one circle, with address_hash deduplication preventing double-insertion. The primary gap risk is at the boundary of the 80% non-overlap zone: if hex-grid alignment is imperfect relative to the drawn polygon boundary, a thin strip of properties may fall outside all sub-circles. This is a low-probability edge case but worth monitoring via the FetchJob's total_properties_found metric.

Self-Recursion Depth & Idempotency

The self-chaining pattern in processFetchChunk is NOT JavaScript call-stack recursion — it is async HTTP invocation via base44.functions.invoke('processFetchChunk', { expected_chunk: nextChunkNumber }), scheduled with a 500ms setTimeout. There is zero stack overflow risk. The mutex check — if (expectedChunk !== null && currentChunkNumber !== expectedChunk) { return Response.json({ skipped: true, reason: 'duplicate_invocation' }) } — prevents race conditions from duplicate invocations. Maximum invocation depth equals the total number of chunks: ceil(totalProperties / 5000). For a 350 sq mile territory at typical residential density (~500 properties/sq mile), this is approximately 35 chunks — well within serverless platform limits.

Error Recovery Gaps

- fetchWithBackoff() retries up to MAX_RETRIES=3 with exponential backoff: Math.min(MAX_BACKOFF_MS, BASE_BACKOFF_MS * Math.pow(2, attempt)) + Math.random() * backoff * 0.3
- 429 handling: reads Retry-After header, falls back to exponential backoff — correct implementation
- 401 errors: immediate return, no retry — correct, prevents credential-burning retry loops
- Network errors: returns { records: [], total: null, status: 0 } — pipeline continues with empty results
- GAP: No dead-letter queue for permanently failed chunks. Failed jobs require manual re-trigger via the UI.

- GAP: fixChristianRoute silently swallows duplicate ValidationQueue errors via .catch(() => {}) — repeated invocations flood the queue with no error signal.
- Recovery block: finds stuck 'running' FetchJobs and marks them status: 'failed' — good defensive pattern

4. Route Optimization Engine

2-opt TSP Implementation Quality

The 2-opt implementation in routeOptimizer.jsx is a genuine, correct implementation — not a simplified approximation. It uses a dummy node strategy to handle open-ended routes and performs standard segment reversal to uncross paths. The core swap logic — const segment = currentRoute.slice(i + 1, j + 1).reverse(); currentRoute.splice(i + 1, segment.length, ...segment) — is textbook 2-opt. The implementation is capped at 50 iterations and falls back to Nearest Neighbor for routes exceeding 300 nodes, which is the correct performance tradeoff for a mobile-first application. The Or-Opt (Link Swap) post-processing step, capped at 30 iterations and 300 nodes, provides additional single-node and 2-node chain relocations that typically yield 30–50% further improvement over raw 2-opt.

Street Sweep Pattern

The orderForStreetSweep() function implements a U-shape block walking pattern: odd-numbered houses ascending on one side of the street, even-numbered houses descending on the return. Streets are sorted by Nearest Neighbor + 2-opt applied only to street centroids (not individual properties). The critical design decision — intentionally NOT applying global 2-opt after street sweep — is architecturally correct and well-documented in the code: 'those destroy the street grouping — causing routes to bounce between streets instead of completing one before the next.' This is the right tradeoff for D2D sales, where completing a full block before moving to the next is operationally superior to a globally optimal but geographically scattered route.

Route Scoring Formula

The competitiveness score formula combines average property score, route efficiency (properties per mile), density bonus, and commute penalty:

Table 7 — Route Competitiveness Score Formula Breakdown

Component	Formula	Weight / Cap	Notes
Average Property Score	avgScore	60% weight	Weighted sum of property attributes from scoreProperty()
Route Efficiency	efficiency * 100	40% weight	orderedProps.length / Math.max(totalDistance, 0.1) — 0.1 prevents division by zero
Density Multiplier	1 + Math.min(0.2, (maxDensity / 10) * 0.2)	Up to +20% bonus	H3 hex density bonus for high-concentration areas
Commute Penalty	Math.min(distanceFromStart * 2, 20)	Capped at -20 points	Penalizes routes far from rep's current location
Final Score	Math.round(((avgScore * 0.6 + efficiency * 100 * 0.4) * densityMultiplier) - commutePenalty)	Integer	Used for route ranking and auto-assignment

Time-of-Day Optimization — Integration Gap

The optimizeRouteForTime() function in knockTimeOptimizer.jsx is a well-designed time-aware property sorter. It maps properties to time scores based on BASE_TIME_SCORES (weekday peak: 18:00 = 95, 19:00 = 90; Saturday peak: 11:00 = 90), adds bonuses for recently sold properties (+20 for < 30 days, +10 for < 90 days) and high-value homes during business hours. However, a critical integration gap exists: optimizeRouteForTime() is exported but never imported or called from generateOptimizedRoutes(). It is a standalone utility that callers must invoke separately — meaning the time-of-day optimization is silently not applied during route generation unless explicitly called by the UI layer.

Table 8 — knockTimeOptimizer.jsx BASE_TIME_SCORES (Selected Hours)

Hour	Weekday Score	Saturday Score	Sunday Score
8:00	15	40	5
9:00	20	70	10
10:00	25	85	15
11:00	30	90	20
12:00	35	75	30

14:00	40	55	55
16:00	60	70	65
17:00	85	75	60
18:00	95	65	50
19:00	90	50	40
20:00	70	30	25

Multi-Rep Collision Avoidance — Missing Feature

No WebSocket code exists in routeOptimizer.jsx. No multi-rep collision avoidance is implemented at the route generation level. Two reps can be assigned overlapping routes simultaneously with no conflict detection. The InteractionLog WebSocket subscription in RepHome.jsx prevents double-knocking within an active session, but does not prevent two reps from being dispatched to the same territory.

Fatigue-Aware Front-Loading

The FATIGUE_FRONT_LOAD_STOPS= 22 constant limits the fatigue-aware front-loading algorithm to move high-propensity stops (top FRONT_LOAD_PROPENSITY_PERCENTILE = 0.30, i.e., top 30%) into the first 22 positions of the route. This ensures reps knock the highest-value doors before fatigue sets in, provided the reordering does not increase total route distance by more than DEFAULT_MAX_DISTANCE_INCREASE= 0.12 (12%). The turn penalty heuristic (180-degree turns penalized at 0.5 virtual miles, 45-degree threshold) further reduces physical fatigue by minimizing backtracking.

5. Offline-First Architecture

localforage / IndexedDB Usage

The mobile application uses localforage 1.10.0 for offline-first IndexedDB storage. Routes are cached under the key cached_routes; properties are cached under cached_props_{routeId}. The syncOfflineQueue/entry.ts cloud function processes queued interactions when connectivity restores. The server correctly stamps synced_at: new Date().toISOString() server-side to prevent client clock manipulation, and enforces tenant isolation by overwriting the client-provided user with the authenticated token: created_by: user.email.

Sync Queue Robustness Assessment

Table 9 — syncOfflineQueue Robustness Assessment

Feature	Status	Notes
Batch size limit (500 items)	Implemented	Returns HTTP 413 if exceeded — correct
Server-side timestamp stamping	Implemented	synced_at: new Date().toISOString() — prevents clock manipulation
Tenant isolation	Implemented	created_by: user.email overwrites client value — correct
True idempotency key	MISSING	Relies on SDK bulkCreate constraints — duplicates possible on retry
successLog / errorsLog tracking	Dead Code	Arrays declared but never populated — no sync observability
Conflict resolution for reassigned routes	MISSING	If route reassigned server-side while rep is offline, sync will conflict

The code comment in syncOfflineQueue/entry.ts explicitly acknowledges the idempotency gap: 'With Base44 SDK, we use bulkCreate and handle duplicates via the SDK constraints.' This is fragile. Mobile network retries are common, and duplicate InteractionLog entries will corrupt rep performance metrics and ML training data. Implement a dedicated idempotency_key field (UUID generated client-side at knock time) with a server-side uniqueness check before bulkCreate.

Unencrypted PII in IndexedDB

IndexedDB is unencrypted at rest on both Android and iOS. The FirstKnock mobile app stores homeowner names, phone numbers, property addresses, sale prices, and potentially authentication tokens in localforage. This violates basic mobile security hygiene and potentially CCPA/GDPR compliance requirements for PII handling. The fix is to replace plain-text localforage storage with encrypted secure storage — @capacitor/preferences for auth tokens and small key-value data, or a secure SQLite wrapper (e.g., @capacitor-community/sqlite with SQLCipher) for the full property cache.

cleanRoute Function — Clarification

cleanRoute/entry.ts is NOT a hash normalization algorithm. It is a manual property status override endpoint. When force_reject === true, it sets original_status = 'REJECTED' and sale_confidence = 'REJECTED'. When force_reject === false, it sets original_status = 'HEURISTIC_SOLD' and sale_confidence = 'medium'. The function is idempotent (only writes if the current DB value differs

from the target), but it uses asServiceRole without verifying that the authenticated user owns the properties being modified. Any authenticated user can override any property's status — a significant data integrity risk.

6. Security & Data Integrity

Critical Vulnerabilities (P0)

Table 10 — P0 Critical Security Vulnerabilities

File	Vulnerability	Impact	Fix
fixChristianRoute/entry.ts	Hardcoded PII + No Auth + asServiceRole	Any unauthenticated HTTP request mutates up to 25,000 records and triggers unbounded BatchData billing. Errors silently swallowed via .catch(() => {}).	DELETE the file immediately. If repair logic is needed, rewrite as an authenticated admin-only function with no hardcoded data.
processReferral/entry.ts	O(n) Database Scan	User.list('-created_date', 5000) fetches entire user table into memory. OOM crashes and timeouts at scale. Users beyond position 5,000 have silently broken referral codes.	Replace with: User.filter({ referral_code: code }) — a single indexed lookup.
autoAssignRoute/entry.ts	Unauthenticated Endpoint	No auth check before parsing payload. Attacker can spoof route.assigned_to or route.start_location to manipulate assignment algorithm and extract internal route data.	Add internal service token validation. Validate payload signature before processing.

High Severity Vulnerabilities (P1)

- processReferral/entry.ts — TOCTOU Race Condition: const newBalance = (referrer.referral_balance || 0) + commission; followed by User.update() is a classic read-modify-write race. Concurrent webhook calls overwrite each other. Fix: use atomic increment at the database level.

- processReferral/entry.ts — Broken Stripe Webhook Auth: The file begins with const user = await base44.auth.me(). Stripe webhooks send cryptographic signatures, not user bearer tokens. This endpoint will always return 401 when called by Stripe. Fix: use Stripe SDK to verify the Stripe-Signature header.
- leadScoring.jsx — Client-Side Scoring: Implementing multi-factor lead scoring on the client exposes proprietary scoring weights and API keys to end users. Malicious reps can reverse-engineer weights to inflate their territory's scores, poisoning the ML training pipeline. Fix: migrate scoring to a Deno backend function; client receives only the final computed score.
- GpsTracker.jsx — GPS Spoofing: No server-side velocity checks between consecutive knock coordinates. No device attestation (Play Integrity API on Android, DeviceCheck on iOS). Reps can fake entire routes from home using mock GPS apps.
- cleanRoute/entry.ts — Missing Ownership Check: Uses asServiceRole without verifying the authenticated user owns the properties being modified. Any authenticated user can override any property's status.

Referral Code Security Analysis

Table 11 — Referral Code Security Analysis

Property	Current Implementation	Risk	Recommendation
Format	FK-[A-Z]{0,4}-[A-Z0-9]{5}	Predictable structure aids enumeration	Acceptable format, but increase random portion length
RNG	Math.random().toString(36).substring(2, 7)	NOT cryptographically secure — predictable with sufficient samples	Replace with crypto.getRandomValues() or
Entropy	36^5 = 60,466,176 combinations	Brute-forceable in minutes with no rate limiting	Increase to 8+ random characters: 36^8 = 2.8 trillion combinations
Collision Check	None — no DB lookup before assignment	Birthday Paradox guarantees collisions as user base grows	Add: if (await User.filter({ referral_code: newCode }).length > 0) regenerate
Rate Limiting	None on apply_code action	Attacker can enumerate all codes to steal signup attribution	Add rate limiting: max 5 apply_code attempts per IP per hour

GPS Proof Tamper Resistance

The GPS proof-of-visit system stores gps_proof_lat, gps_proof_lng, and gps_accuracy in every

InteractionLog record. This is the right data model, but the data is entirely untrustworthy without server-side validation. On Android, any app can enable Mock Location in Developer Options — no root required. On iOS, Xcode location simulation works on non-jailbroken devices during development, and jailbroken devices have trivial GPS spoofing tools. The recommended server-side velocity check: calculate Haversine distance between consecutive knocks for the same rep, divide by elapsed seconds, and reject any knock where the implied travel speed exceeds 26.8 m/s (60 mph). Additionally, integrate Capacitor plugins to detect Mock Location settings at the application layer.

autoAssignRoute Data Exposure

The autoAssignRoute/entry.ts function returns the entire bestRoute object in its response: return Response.json({ route: bestRoute }). This leaks all property hashes, lead scores, internal metrics, and potentially homeowner PII to the caller. Since the endpoint has no authentication, this data is publicly accessible to anyone who discovers the endpoint URL. The fix is to return only the assignment confirmation: { assigned: true, route_id: bestRoute.id, assigned_to: targetRep.id }.

7. Performance Bottlenecks

Leaflet CircleMarker Rendering at 50,000 Pins

The Home.jsx manager dashboard loads up to 50,000 MasterProperty records and renders each as a Leaflet CircleMarker. Standard Leaflet uses DOM-based SVG or Canvas rendering — each marker is a DOM element. Browser performance degrades severely above approximately 5,000 DOM markers, and at 50,000 markers the browser will freeze on most consumer hardware. The current architecture chunks ZIP queries into batches of 5 to prevent browser memory overflow, but the rendering bottleneck remains. The recommended fix is to migrate to canvas-based rendering: react-leaflet-cluster for automatic clustering at zoom levels below street view, or deck.gl ScatterplotLayer for GPU-accelerated rendering of the full 50,000-point dataset.

filterByStreetCooldown() Complexity

The filterByStreetCooldown() function in territoryLogic.jsx has O(ntimesm) complexity, where n is the number of properties and m is the number of interaction logs. At 1,000 concurrent reps each with 50 properties and 100 interaction logs, this produces 5 million operations per route generation call. This function must be refactored to use a pre-indexed Map of address_hash rightarrow latest interaction, reducing complexity to O(n + m).

ZIP Batch Parallelism

fetchZipProperties processes RentCast pages in parallel batches of 3 (not 5 as stated in some documentation): for (let i = 0; i < tasks.length; i += 3). The code comment notes this is 'conservative to avoid rate limits.' Increasing to 5 parallel pages would improve throughput by

approximately 67% if RentCast's rate limits allow. RentCast's standard plan supports 100 requests/minute — at 300ms sleep between batches and 3 parallel requests, the current throughput is approximately 10 req/s, well below the rate limit ceiling.

TanStack Query Cache Strategy

The codebase uses @tanstack/react-query 5.84.1 as the client-side cache layer. The optimal cache strategy for this application: staleTime of 5 minutes for MasterProperty data (property records change infrequently), 30 seconds for SavedRoute data (route status changes frequently during active sessions), and immediate cache invalidation on InteractionLog mutations (knock events must propagate to the manager map in real time via the existing WebSocket subscription).

8. Cloud Functions Reference Audit

Idempotency Guarantees by Function

Table 12 — Cloud Function Idempotency Assessment

Function	Idempotent?	Mechanism	Risk
processFetchChunk	Yes	Mutex check (expected_chunk vs job.chunk_number) + hash-based dedup via existingHashToId Map	Low — well-implemented
cleanRoute	Yes	Conditional write only if current DB value differs from target	Low
fetchZipProperties	Partial	address_hash dedup on insert, but BatchData calls are not deduplicated	Medium — duplicate BatchData costs on retry
syncOfflineQueue	No	Relies on SDK bulkCreate constraints — no idempotency_key check	High — mobile retries create duplicate InteractionLog entries
fixChristianRoute	Partial	Status changes are idempotent; ValidationQueue entries are NOT	Critical — repeated invocations flood queue

processReferral	No	TOCTOU race on balance update; no idempotency key on commission credit	High — financial discrepancies under concurrent load
autoAssignRoute	No	No dedup check — same route can be assigned multiple times if automation fires twice	Medium

Deno Runtime Cold Start Analysis

All cloud functions use Deno.serve() on the Base44 serverless platform. Deno cold starts on serverless platforms are typically 50–200ms for lightweight functions. The processFetchChunk function, with its large import set (h3-js 4.1.0, @base44/sdk, @neondb/serverless 0.9.0), may experience cold starts of 300–500ms. The 500ms setTimeout before self-chaining provides adequate buffer. The trainLeadPredictor function, which implements the full Beta-Binomial Bayesian model with ADWIN drift detection, is the most computationally intensive function and should be monitored for timeout risk on large training datasets.

Dependency Risk Assessment

Table 13 — Dependency Risk Assessment

Dependency	Version	Risk	Notes
@neondb/serverless	0.9.0	Medium	Direct PostgreSQL access bypasses Base44 SDK tenant isolation. Must be audited for RLS compliance on every usage.
lodash	4.17.21	Low-Medium	merge() and cloneDeep() are susceptible to prototype pollution if fed unsanitized user input. Audit all lodash.merge() call sites.
h3-js	4.1.0	Low	Stable Uber H3 library. Resolution 9 usage is appropriate for D2D use case.

turf	3.0.14	Low	Outdated — current version is 6.x. Consider upgrading for bug fixes and performance improvements.
@capacitor/core	8.0.0	Low	Web-view based mobile (not React Native). Appropriate for this use case. Ensure Capacitor plugins are audited for permissions.
localforage	1.10.0	Medium	Unencrypted IndexedDB storage. PII at rest violates mobile security hygiene. Replace with encrypted storage for sensitive data.

9. ML System Analysis

Beta-Binomial Bayesian Model (bayesian_v3)

The trainLeadPredictor/entry.tsimplements a Beta-Binomial Bayesian model with ADWIN (Adaptive Windowing) drift detection. The prior configuration — PRIOR_ALPHA = 2, PRIOR_BETA = 18 — establishes a 10% base conversion rate assumption (alpha / (alpha + beta) = 2/20 = 0.10). This is a reasonable prior for D2D sales conversion rates. The graduated outcome weights (SOLD: 1.0, QUALIFIED: 0.8, CALLBACK: 0.4, NO_ANSWER: 0.0, HARD_NO: -0.3) are more sophisticated than binary classification and correctly reflect the partial-value nature of callbacks and qualified leads.

Table 14 — ML Model Configuration Analysis

Component	Value / Detail	Assessment
Model Type	Beta-Binomial Bayesian (bayesian_v3)	Appropriate for conversion rate estimation with small sample sizes
Prior Alpha / Beta	2 / 18 (10% base conversion rate)	Reasonable for D2D sales. Slightly conservative — adjust if actual conversion rate differs significantly.

Features (6 total)	age_gt_10, price_gt_300k, single_family, recent_sale, high_value, large_lot	Feature set is sparse. Adding ownership_tenure and neighborhood_velocity would improve accuracy.
ADWIN Min Window	30 observations	Appropriate minimum for statistical significance
ADWIN Epsilon Cut	0.15 (15% divergence threshold)	Conservative — will only trigger on significant market shifts
ADWIN Split Testing	30% to 70% in 5% steps	Thorough split testing reduces false drift detection
Channel Weights	ownership: 0.35, heat: 0.25, pqi: 0.20, distress: 0.20	Ownership signal correctly weighted highest for D2D solar/home services
Training Data Integrity	At risk — client-side leadScoring.jsx can be manipulated	If scoring is client-side, training pipeline ingests untrusted data

ADWIN_CONFIDENCE = 0.95 is defined as a constant in the scoring engine but is never referenced in the adwinTrimWindow() function. The drift detection logic uses only ADWIN_EPSILON_CUT = 0.15 for divergence comparison. This unused constant suggests either incomplete implementation or a refactoring artifact. Clarify intent and either integrate the confidence parameter or remove the dead constant.

10. Optimization Recommendations

The following table consolidates all identified issues with priority, estimated effort, and remediation guidance. Issues are ordered by priority (P0 rightarrow P3) and then by estimated effort within each priority tier.

Table 15 — Complete Optimization Recommendations (Ordered by Priority)

#	Issue	File / Location	Impact	Priority	Est. Effort

1	fixChristianRoute in production — hardcoded PII (christian@nativapest.com), no auth, asServiceRole, can mutate 25K records, silently swallows queue errors via .catch(() => {})	fixChristianRoute/entry.ts	Critical — financial & data integrity	P0	1 hour
2	O(n) user scan — User.list('-created_date', 5000) full table scan to find referral code; OOM at scale; users beyond 5,000 have silently broken	processReferral/entry.ts	Critical — availability	P0	2 hours
3	GPS spoofing — no server-side velocity checks between consecutive knocks; mock GPS apps trivially bypass proof-of-visit	GpsTracker.jsx + knock verification endpoint	High — core value proposition	P1	8 hours

4	Referral code uses Math.random() — non-CSPRNG, brute-forceable 60M combinations, no rate limiting on apply_code, no collision	processReferral/entry.ts	High — attribution fraud	P1	2 hours
5	Race condition in commission crediting — TOCTOU on referral_balance read-modify-write; concurrent calls overwrite each other	processReferral/entry.ts	High — financial discrepancy	P1	4 hours
6	Split-brain address hashing — three incompatible formats (pipe-separated, three-part pipe, kebab-case) cause cache misses and duplicate	processFetchChunk, routeOptimizer, migrateHashLegacy	High — data integrity	P1	16 hours
7	cleanRoute uses asServiceRole without ownership check — any authenticated user can override any property's status	cleanRoute/entry.ts	High — data integrity	P1	4 hours

8	autoAssignRoute leaks full route object in response — exposes all property data, lead scores, and PII to unauthenticated callers	autoAssignRoute/entry.ts	Medium — data exposure	P1	1 hour
9	syncOfflineQueue no idempotency key — mobile retries create duplicate InteractionLog entries; successLog/errorsLog are dead code	syncOfflineQueue/entry.ts	Medium — data integrity	P2	8 hours
10	optimizeRouteForTime() not integrated — exported from knockTimeOptimizer.jsx but never called from generateOptimizedRoutes()	knockTimeOptimizer.jsx	Medium — feature gap	P2	4 hours
11	Leaflet performance at 50K markers — DOM-based CircleMarker rendering freezes browser above ~5K pins	Home.jsx + map components	High — UX	P2	16 hours

12	No multi-rep collision avoidance — two reps can be assigned overlapping routes with no conflict detection	routeOptimizer.jsx + autoAssignRoute	Medium — operational	P2	24 hours
13	Missing denormalized fields on MasterProperty — homeowner_equity, ownership_tenure_years, neighborhood_velocity computed at runtime	MasterProperty schema	Medium — performance	P2	40 hours
14	No dead-letter queue for failed chunks — permanently failed processFetchChunk invocations	processFetchChunk/entry.ts	Medium — reliability	P2	8 hours
15	Heuristic scoring inconsistency — listingDuration < 7 penalty is -1 in processFetchChunk vs -2 in fetchZipProperties	processFetchChunk vs fetchZipProperties	Medium — data quality	P2	4 hours

16	Unencrypted PII in IndexedDB — localforage stores homeowner data and potentially auth tokens unencrypted at rest	storage.js	Medium — compliance	P2	16 hours
17	autoAssignRoute fallback lat:0,lng:0 for missing location — silent fallback to null island coordinates corrupts distance calculations	autoAssignRoute/entry.ts	Low — data quality	P2	2 hours
18	Weather stub not implemented — getWeatherModifier() returns hardcoded values for string conditions; no real API integration	knockTimeOptimizer.jsx	Low — feature gap	P2	16 hours
19	BatchData cap discrepancy — FetchJob.jsonc schema says 500, code enforces 100 via allowedMisses.slice(0, 100)	processFetchChunk/entry.ts + FetchJob.jsonc	Low — documentation	P3	1 hour

20	ADWIN_CONFIDENCE = 0.95 unused — constant defined but never referenced in adwinTrimWindow() drift detection logic	trainLeadPredictor/entry.ts	Low — dead code	P3	2 hours
21	FetchJob maxPulls = 9999 left in production — 'unlimited for testing' comment; should be bounded in production config	FetchJob config	Low — safety	P3	1 hour

11. Conclusion & Remediation Roadmap

Architectural Maturity Assessment

The FirstKnock Sales OS demonstrates genuine architectural maturity in its core data pipeline and route optimization engine. The three-step validation pipeline with CDC delta-pulls, heuristic gating, and BatchData cost controls reflects sophisticated engineering judgment. The Beta-Binomial Bayesian ML model with ADWIN drift detection, the 2-opt TSP solver with street-sweep pattern, and the H3 spatial indexing are all production-quality implementations that would be at home in a well-funded Series A company's codebase. The offline-first mobile architecture with localforage sync is well-conceived, and the real-time WebSocket subscription for knock events is the right pattern for multi-rep coordination.

The critical gaps are concentrated in two areas: security hygiene around one-off production scripts (fixChristianRoute, processReferral, autoAssignRoute) and data integrity around the split-brain address hashing strategy. These are not architectural flaws — they are the predictable artifacts of a fast-moving startup that built production infrastructure alongside prototype scripts and never cleaned up. The remediation path is clear, well-scoped, and achievable within two focused engineering sprints. None of the P0 or P1 issues require architectural rewrites; they are targeted fixes to specific files.

2P0 Issues Requiring Immediate Action	**6**P1 High-Severity Issues (Sprint 1)	**10**P2 Medium Issues (Sprint 2)	**3**P3 Low-Priority Cleanup (Sprint 3)

Recommended 3-Sprint Remediation Roadmap

Table 16 — 3-Sprint Remediation Roadmap

Sprint	Timeline	Focus	Key Deliverables	Est. Total Effort
Sprint 1	Weeks 1–2	P0 Elimination + P1 Security	1. DELETE fixChristianRoute/entry.ts2. Replace O(n) user scan with User.filter({ referral_code: code })3. Add server-side GPS velocity checks (reject > 60 mph between knocks) 4. Replace Math.random() with crypto.getRandomValues() in processReferral5. Fix TOCTOU race condition with atomic DB increment6. Standardize address hashing on single canonical format7. Add ownership check to cleanRoute8. Strip full route object from autoAssignRoute response	~38 hours

Sprint 2	Weeks 3–4	P1 Data Integrity + P2 Performance	1. Implement idempotency_key in syncOfflineQueue 2. Integrate optimizeRouteForTime() into generateOptimizedRoutes()3. Migrate Leaflet to canvas-based rendering (react-leaflet-cluster or deck.gl)4. Implement multi-rep territory collision detection 5. Add dead-letter queue for failed processFetchChunk invocations6. Fix heuristic scoring inconsistency (listingDuration < 7 penalty)7. Replace localforage with encrypted secure storage for PII8. Fix null island fallback in autoAssignRoute	~78 hours

Sprint 3	Weeks 5–6	P2 Feature Completions + P3 Cleanup	1. Denormalize homeowner_equity, ownership_tenure_years, neighborhood_velocity onto MasterProperty2. Integrate real weather API into knockTimeOptimizer.jsx3. Update FetchJob.jsonc schema to reflect actual 100-call BatchData cap4. Remove or integrate ADWIN_CONFIDENCE constant5. Bound FetchJob maxPulls in production config 6. Audit all @neondb/serverless direct DB access for RLS compliance7. Upgrade turf from 3.0.14 to 6.x8. Implement TanStack Query cache strategy (staleTime tuning)	~84 hours

IMMEDIATE ACTION (Before Sprint 1 Begins): fixChristianRoute/entry.ts must be removed from production today. It is an unauthenticated endpoint that can trigger unbounded BatchData API billing and mutate 25,000 records. This is not a sprint item — it is an emergency hotfix that should be deployed within hours of this report being received.

Final Recommendation

The FirstKnock Sales OS is a technically impressive platform with a clear product vision and a sophisticated core architecture. The identified vulnerabilities are serious but remediable. The P0 issues (fixChristianRoute and the O(n) user scan) can be resolved in a single afternoon. The P1 security issues (GPS spoofing, referral code entropy, TOCTOU race, split-brain hashing) require focused but bounded engineering work. With the three-sprint roadmap above, the platform can reach production security and data integrity standards within six weeks while continuing to ship

product features in parallel. The underlying architecture — the three-step validation pipeline, the Bayesian ML model, the 2-opt route optimizer — is a strong foundation worth protecting with the security and reliability improvements outlined in this report.

This report was prepared based on static analysis of decoded source files including processFetchChunk/entry.ts, fetchZipProperties/entry.ts, fetchAreaProperties/entry.ts, trainLeadPredictor/entry.ts, autoAssignRoute/entry.ts, processReferral/entry.ts, fixChristianRoute/entry.ts, cleanRoute/entry.ts, syncOfflineQueue/entry.ts, routeOptimizer.jsx, territoryLogic.jsx, knockTimeOptimizer.jsx, GpsTracker.jsx, storage.js, AuthContext.jsx, leadScoring.jsx, and the FetchJob.jsonc schema. Dynamic analysis, penetration testing, and load testing were not performed and are recommended as follow-on activities.